

0.60.2. Aller et revenir : aller_retour — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

1. Ce qu'il faut comprendre

C'est le 0.60 et le 0.60.1 réunis : la même forme à deux places, l'une après l'autre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <aller_retour>` → [0.59.3. La fonction aller_retour\(\) — aller et revenir](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1338 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
aller_retour() 400 o ← main() – une fonction fournie par le langage
main()         67 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()      x 3   39 o sur place, dans aller_retour() + AdresseCase (10 o)
effacer()    x 2   30 o sur place, dans aller_retour() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 3    9 o sur place, dans aller_retour(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
aller_retour() x 2   60 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <aller_retour> // une tuile qui va et revient
#include <ALPHABET>     // les lettres de la police : ALPHABET[0] est le A
```

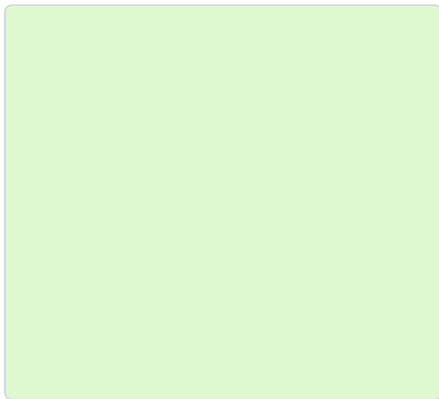
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {           // Le programme commence ici.
    // aller_retour(x, y, tuile, pasX, pasY, vitesse, fois) : de (x, y) à
    // (x + pasX, y + pasY) et retour ; ici 5 cases à droite, 250 ms, 2 fois.
    // 1. Le A, au milieu : de (10, 8) à (15, 8)
    aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);
    // 2. Le B, en haut à gauche : de (4, 4) à (9, 4) – quand le A a fini.
    aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2);

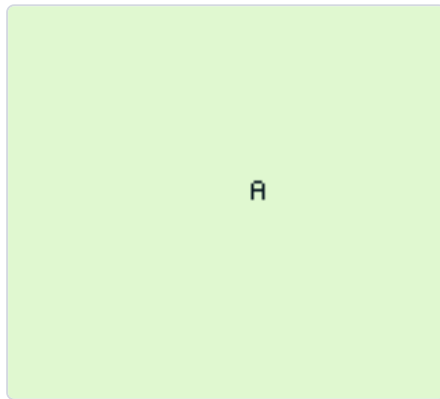
    while (true) {      // La boucle de jeu, pour toujours :
        image();         // attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



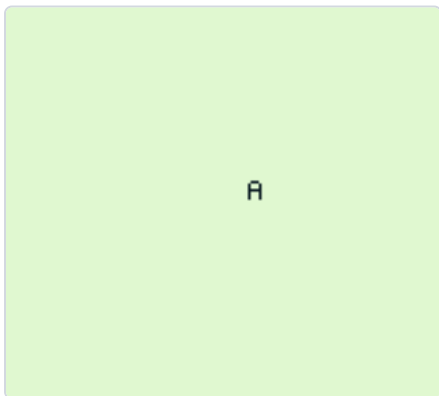
la toute première image — la console vient de démarrer — image 1



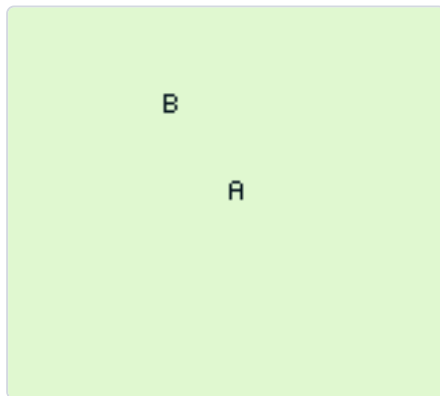
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La forme du A au milieu, puis celle du B en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (10, 8) (10,8)

✓ le B finit en (4, 4) (4,4)

Ensuite : **0.61. Aller et revenir en diagonale** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++